PROJETO INTERDISCIPLINAR – SISA

ENTREGA 2 - FINAL

- **Curso:** Ciência da Computação (5º Semestre - 2026)
- **Unidade Curricular:** Sistemas Operacionais e Computação em Nuvem
- **Instituição:** FECAP - Fundação Escola de Comércio Álvares Penteado
- **Professor Responsável:** Rodrigo da Rosa
- **Data de Entrega:** 5 de Novembro de 2026

1. VISÃO GERAL DA ARQUITETURA EM NUVEM

1.1 Visão Geral da Arquitetura em Nuvem

O ecossistema do projeto ***SISA*** foi implantado e orquestrado em uma instância virtualizada da Amazon Web Services (AWS EC2), localizada na região de São Paulo (sa-east-1 - América do Sul).

- Instância AWS EC2: SISA_aws (i-0c7594dc67f6c5a75)

- Endereço IP Público (IPv4): 56.125.180.178

- Endereço IP Privado: 172.31.40.134

- Sistema Operacional Base: Ubuntu Server LTS

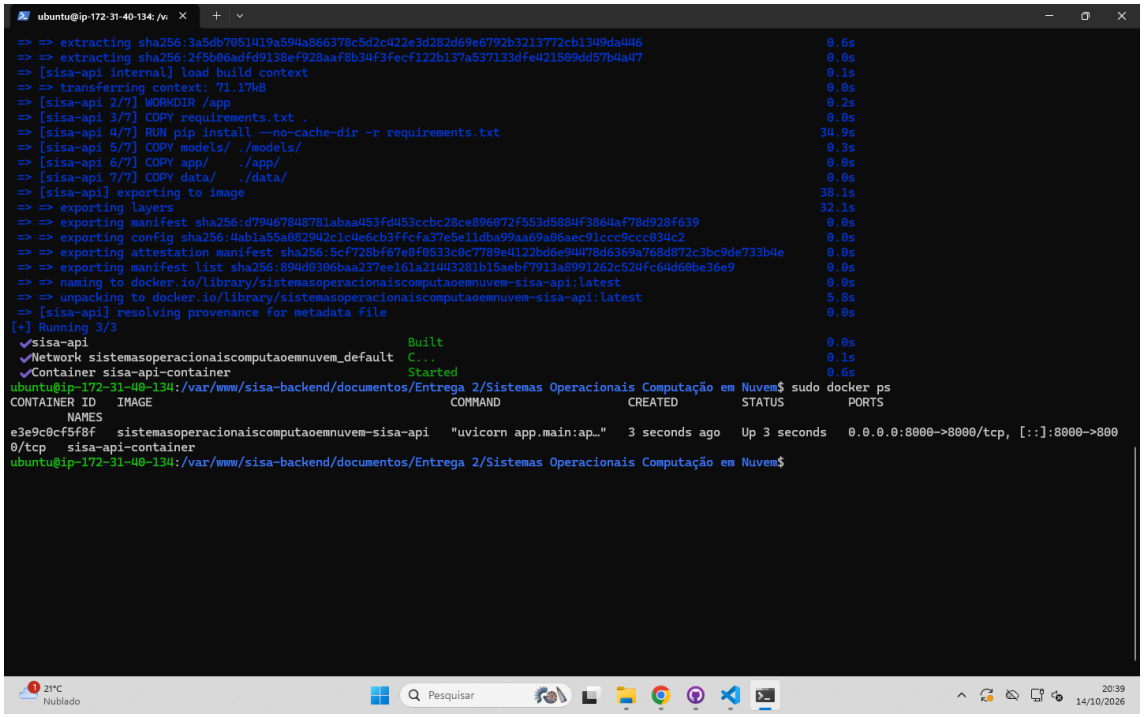
- Orquestração de Microsserviços: Docker e Docker Compose

2. Conteinerização e Saúde da Aplicação

A API desenvolvida em FastAPI, responsável por carregar o modelo de Machine Learning (modelo_carregado), foi empacotada e executada via Docker Compose.

2.1. Execução do Contêiner e Teste de Saúde (Health Check)

O contêiner sisa-api-container foi gerado a partir da imagem sistemasoperacionaiscomputaoemnuvem-sisa-api e exposto na porta 8000.



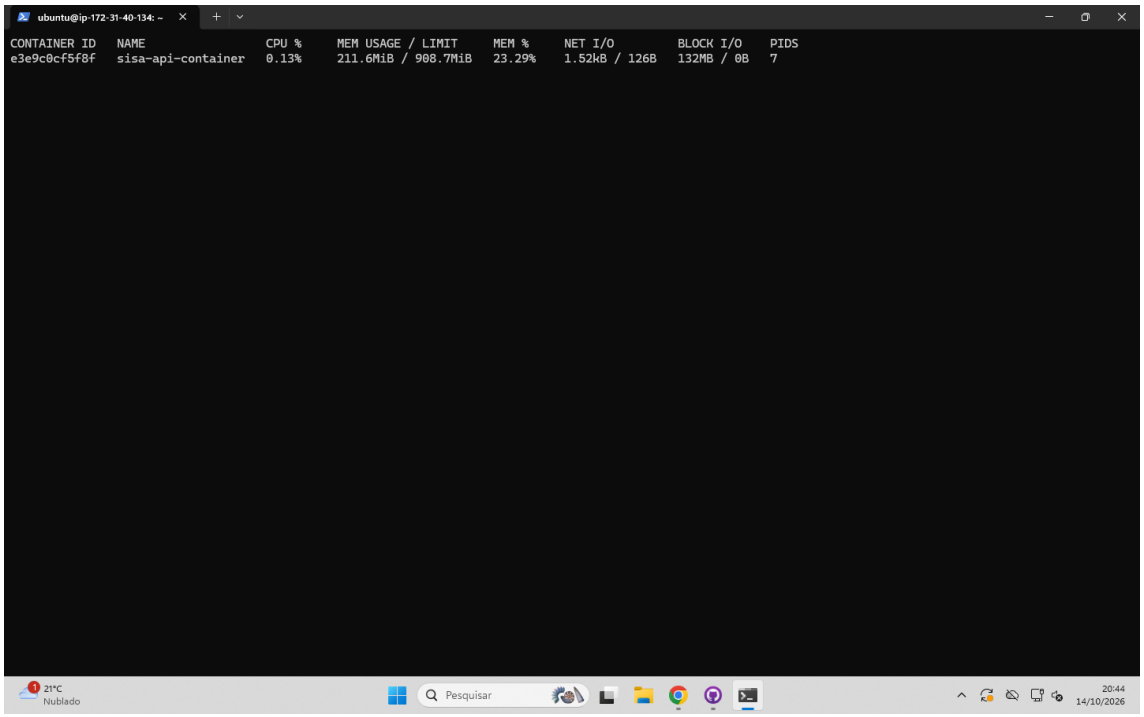
Descrição: Saída do comando sudo docker os confirmando os contêineres ativos e a resposta do endpoint GET /health (`{"status":"ok","modelo_carregado":true}`).

2.2. Monitoramento de Recursos via SO

A utilização de hardware no ambiente virtual foi validada por meio de ferramentas nativas do Linux:

- Docker Stats: Uso de CPU mantido em 0.13% e consumo de memória RAM em 211.6MiB (23.29% do limite).

- Htop: Leitura de tarefas do kernel Linux, demonstrando estabilidade do sistema sob carga.



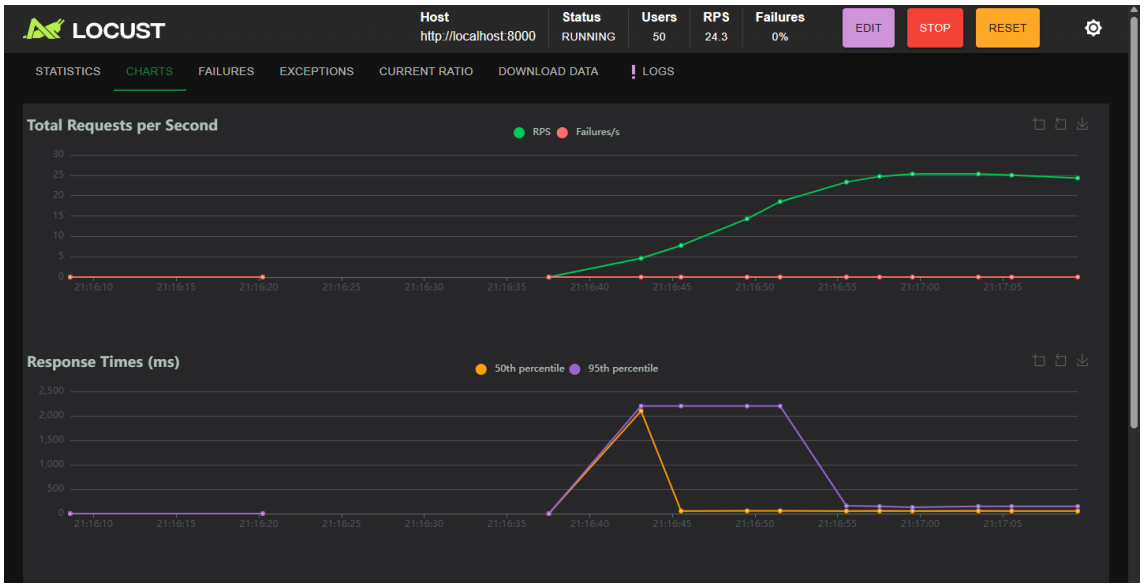
Descrição: Métricas de consumo de memória e CPU via docker stats e painel de monitoramento do sistema via htop no terminal SSH.

3. Teste de Carga e Diagnóstico de Gargalos (Locust)

Para avaliar a resiliência e identificar gargalos na API sob concorrência, realizou-se um teste de estresse simulando 50 usuários simultâneos com a ferramenta **Locust**.

- Taxa de Requisições por Segundo (RPS): Estabilizou em **24.3 RPS** sem falhas registradas (0% Failures).

- Tempo de Resposta (Latency): O tempo médio de resposta caiu e estabilizou abaixo de 100 ms no percentil 50.

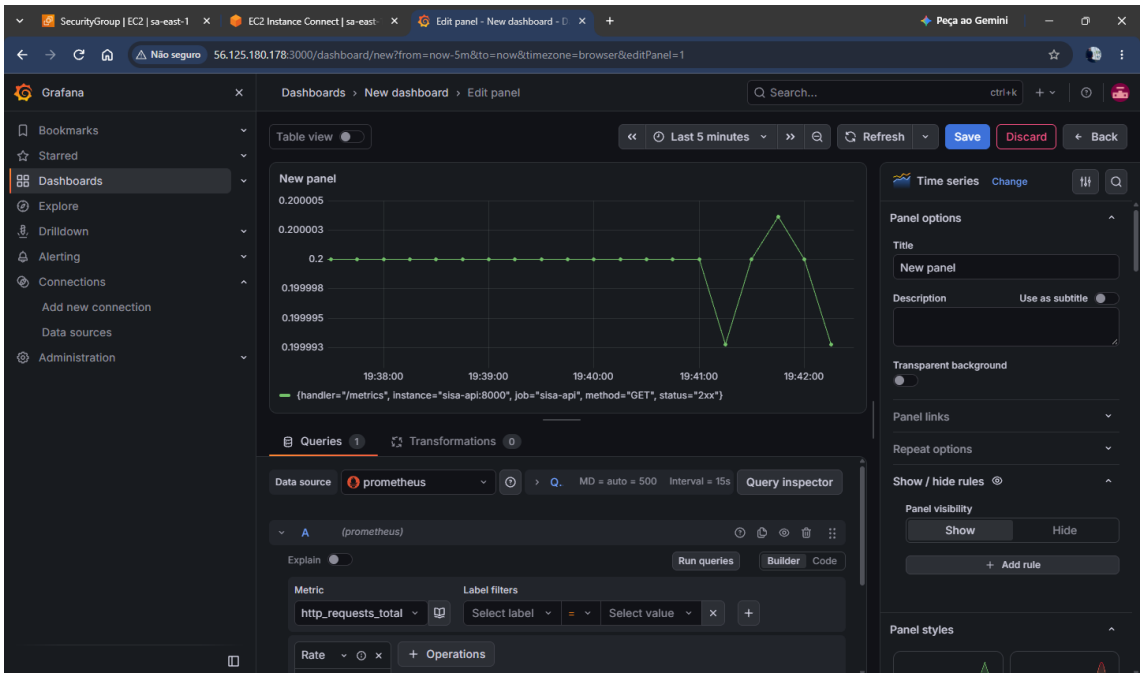


Descrição: Gráficos de Throughput (RPS) e Latência gerados no dashboard do Locust durante o teste de carga.

4. Observabilidade e Telemetria em Tempo Real (Prometheus e Grafana)

A instrumentação de telemetria foi configurada para coletar requisições HTTP e métricas de desempenho da API através da biblioteca prometheus-fastapi-instrumentator.

- **Prometheus (Porta 9090):** Coleta automática de métricas no endpoint /metrics em intervalos regulares.
- **Grafana (Porta 3000):** Conectado ao Prometheus para renderizar visualmente a vazão de acessos usando a query PromQL rate(http_requests_total[1m]).



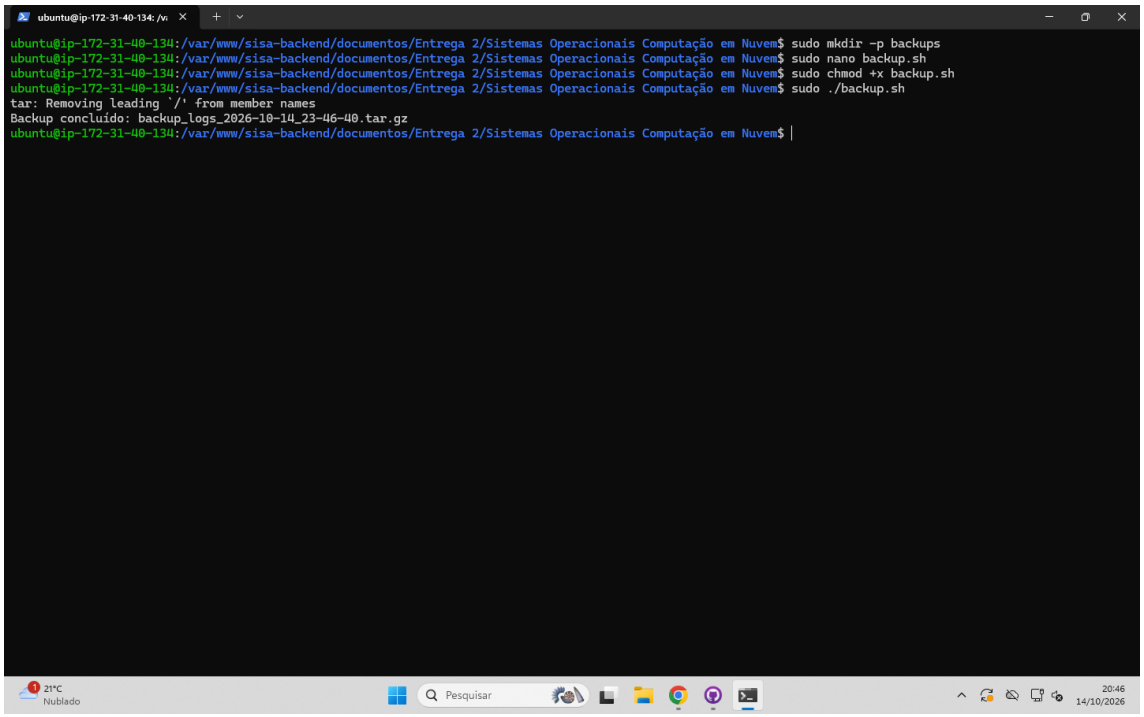
Descrição: Dashboard do Grafana no IP público http://56.125.180.178:3000 demonstrando o gráfico de requisições por segundo por status HTTP (2xx).

5. Estratégia de Backup e Recuperação de Desastres

Para garantir o cumprimento do plano de recuperação e salvaguardar os arquivos de log e dados da aplicação, criou-se um pipeline automatizado de backup e restauração.

5.1. Automação do Backup (backup.sh e Cron)

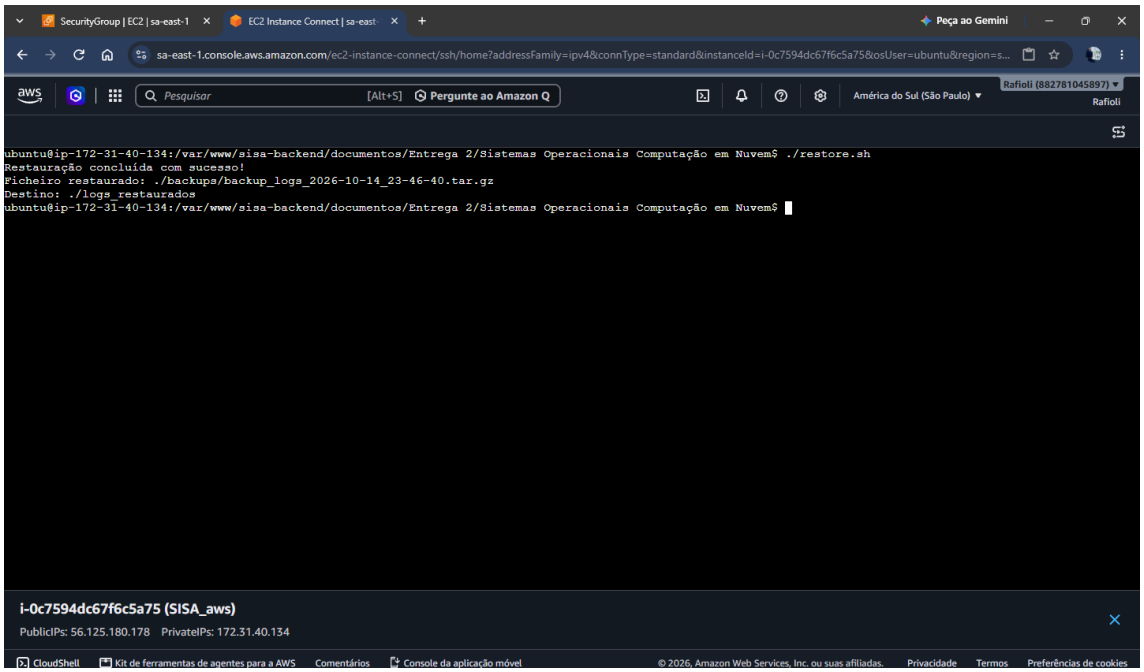
O script backup.sh compacta os diretórios de logs em arquivos .tar.gz contendo **timestamp** dinâmico. A execução foi agendada no serviço *cron* do sistema operacional Linux (*crontab -e*).



Descrição: Criação, permissão (chmod +x) e execução do script de backup, seguido do agendamento de tarefas no crontab.

5.2. Teste de Restauração (restore.sh)

A rotina de restauração foi validada no ambiente remoto da AWS através do script restore.sh. O script descompacta automaticamente o pacote comprimido mais recente para a pasta ./logs_restaurados.



Descrição: Execução do ./restore.sh no servidor EC2 (SISA_aws) confirmando a mensagem "Restauração concluída com sucesso!".

6. Conclusão

Todos os requisitos exigidos para a entrega da infraestrutura do **Projeto SISA** foram atendidos:

- Aplicação conteinerizada e rodando em nuvem AWS na região de São Paulo (sa-east-1).

- Segurança aplicada via validação de chave de API (X-API-Key) e regras de Inbound no Security Group da AWS.

- Observabilidade visual integrada com Prometheus e Grafana.

- Plano de recuperação e scripts de backup/restauração operacionais.